Experiment tracking

With MLflow

Last session

Every pull request runs tests and lint. A red check blocks the merge.

CI answers **"does it run?"**. It does not answer:

- Which parameters gave the best model?
- Which data was it trained on?
- Which model is the one in production, and how do I load it?

Today: keep track of every training run, and give models a name.

Today

Part 1: Reproducibility. What you need to rerun an experiment

Part 2: MLflow tracking, hands-on. Log runs, compare them in the UI

Part 3: A shared tracking server with Docker Compose, artifacts in the bucket

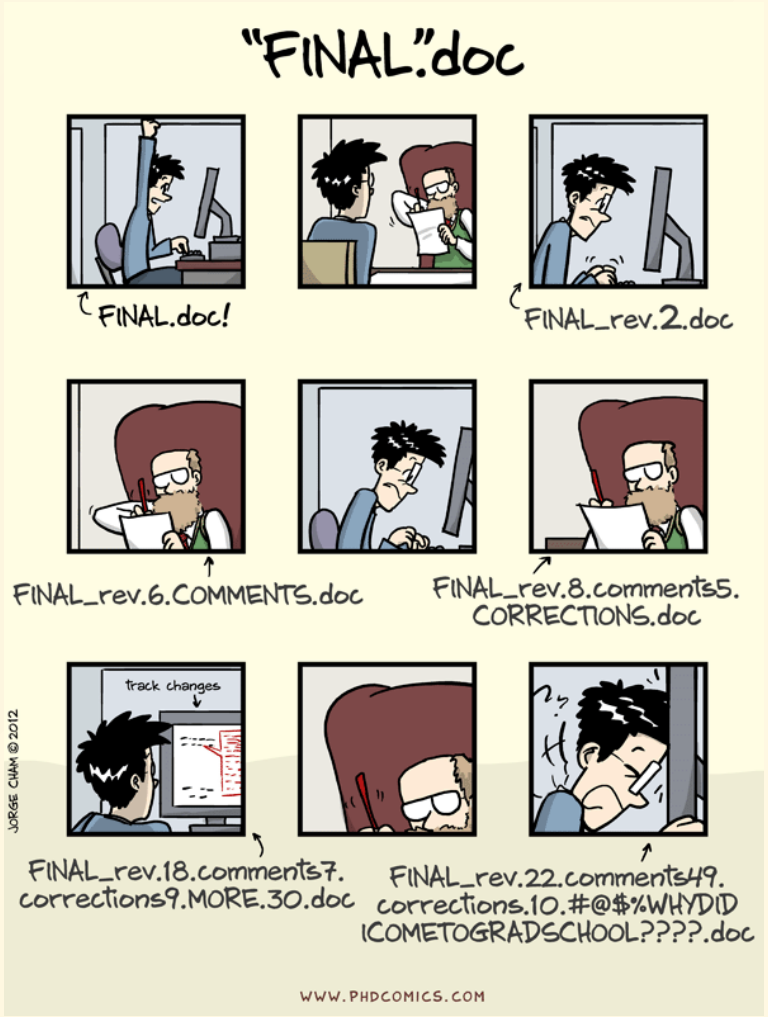
Part 4: Model Registry. Versions, aliases, loading and serving a model by name

Part 5: Data versioning, briefly: DVC

Part 1

Reproducibility

Version Control



What makes a result reproducible?

To get the same model again you need the same:

What	Where it lives
Code	Git (class 1)
Environment	`requirements.txt`, Docker (class 2)
Data	Bucket, Parquet (class 3), DVC
Parameters	? (Today)
Results	? (Today)

The notebook problem

A week of experiments usually ends like this:

```
model.pkl
model_v2.pkl
model_v2_depth10.pkl
model_final.pkl
model_final_REAL.pkl
results.xlsx      # "depth 10 was better, I think"
```

- Which code and which data produced `model_final_REAL.pkl`?
- Was Mean Absolute Error in the xls computed on same test set?
- Your teammate got a better result. With which parameters?

Experiment tracking

Write down **every run**, **automatically**, in one place:

- **Parameters**: what you chose (`max_depth=10`)
- **Metrics**: how well it did (`mae=3.44`)
- **Artifacts**: what it produced (the model, plots, files)
- **Metadata**: when, who, which Git commit, how long

Then compare runs side by side, and pick a model with evidence.

Part 2

MLflow tracking

What is MLflow?

Open source platform for ML lifecycle. Python library + server + UI.

Today we use three parts of it:

- **Tracking**: log parameters, metrics and artifacts of each run
- **Models**: a standard way to save a model with its dependencies
- **Model Registry**: named models with versions and aliases

Works with scikit-learn, PyTorch, XGBoost, … and runs anywhere: your laptop, a container, a cloud service.

Experiments and runs

```
Experiment: trip-duration
├── Run rf-depth-2     params: max_depth=2    metrics: mae=4.52
├── Run rf-depth-5     params: max_depth=5    metrics: mae=3.81
├── Run rf-depth-10    params: max_depth=10   metrics: mae=3.44
└── Run rf-depth-20    params: max_depth=20   metrics: mae=3.32
                       artifacts: model/
```

- **Experiment**: a group of runs for the same problem
- **Run**: one execution of your training code with
  - parameters used
  - metric results like mean absolute error.

The problem

Same taxi trips from the storage session. Predict how long a trip takes:

- **Target**: `duration_min`, dropoff time minus pickup time
- **Features**: `trip_distance`, `pickup_hour`, `pickup_weekday`, `PULocationID`, `DOLocationID`
- **Model**: scikit-learn `RandomForestRegressor`
- **Metric**: MAE, mean absolute error, in minutes

100,000 trips sampled from January, so a run takes seconds.

Install

Add to `requirements.txt`:

```
mlflow==3.17.2
scikit-learn==1.9.1
```

Activate your virtualenv and install.

```
pip install -r requirements.txt
```

MLflow 3.17 needs `pyarrow<26`: pip picks `pyarrow` 25 for you.

Note: We had pyarrow==26.0.0 pinned, remove the pin or pin to 25.

Preparing the data: features.py

```
import pandas as pd
from sklearn.model_selection import train_test_split

FEATURES = ["trip_distance", "pickup_hour", "pickup_weekday",
            "PULocationID", "DOLocationID"]

def load_trips(path="data/trips.parquet", n=100_000):
    df = pd.read_parquet(path)
    df["duration_min"] = (
        df["tpep_dropoff_datetime"] - df["tpep_pickup_datetime"]
    ).dt.total_seconds() / 60
    df["pickup_hour"] = df["tpep_pickup_datetime"].dt.hour
    df["pickup_weekday"] = df["tpep_pickup_datetime"].dt.weekday
    # Drop trips of less than a minute or more than two hours
    df = df[df["duration_min"].between(1, 120)]
    df = df.sample(n, random_state=42)
    X = df[FEATURES].astype("float64")
    return train_test_split(X, df["duration_min"],
                            test_size=0.2, random_state=42)
```

Your first run: train.py

```
import mlflow
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error

from features import load_trips

X_train, X_test, y_train, y_test = load_trips()

mlflow.set_experiment("trip-duration")

with mlflow.start_run():
    params = {"n_estimators": 50, "max_depth": 5}
    mlflow.log_params(params)

    model = RandomForestRegressor(**params, random_state=42)
    model.fit(X_train, y_train)
```

Your first run: train.py

```
    mae = mean_absolute_error(y_test, model.predict(X_test))
    mlflow.log_metric("mae", mae)
    mlflow.sklearn.log_model(
        model,
        name="model",
        input_example=X_test[:5],
        # skops only loads types it knows are safe. Tree models must
        # be trusted explicitly: only for models you trust.
        skops_trusted_types=["sklearn.tree._tree.Tree"],
    )
    print(f"MAE: {mae:.2f} minutes")
```

```
python train.py
```

```
MAE: 3.81 minutes
```

Where did it go?

Nothing to configure. By default MLflow writes next to your code:

```
mlflow.db      # parameters, metrics, run metadata (SQLite)
mlruns/        # artifacts: the model files
```

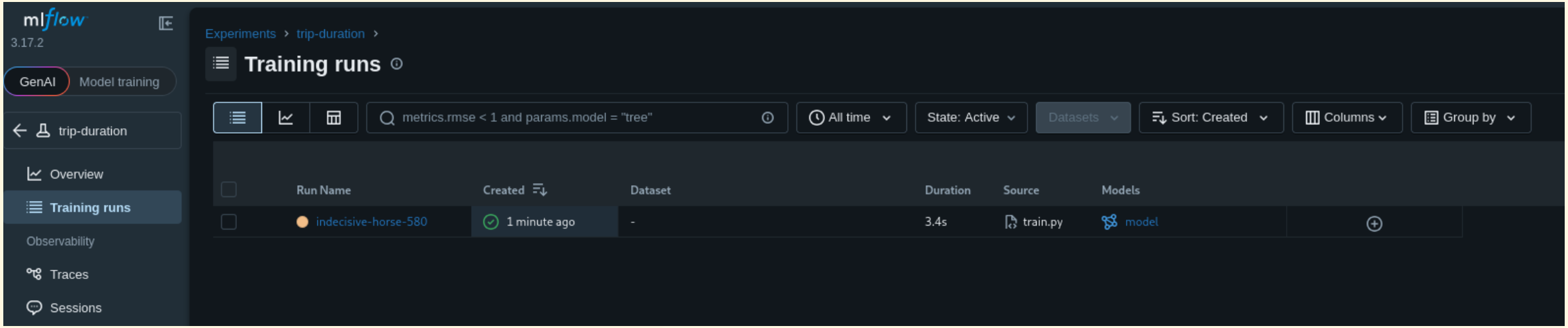
Add both to `.gitignore`. Then start the UI:

```
mlflow ui --port 5050
```

Open http://localhost:5050

The default port is 5000 but you can redefine, in this case 5050.

Your first run in the UI



What a run contains

Click on the run:

- **Parameters**: `n_estimators`, `max_depth`
- **Metrics**: `mae`
- **Artifacts**: the model with its `MLmodel` file, `requirements.txt`, `conda.yaml` and the input example
- **Tags**: user, source file, Git commit (if you run it inside a repo)

The model is saved with **what it needs to run**: the Python version and the libraries.

Exercise 1 - README - first run

1. Add `mlflow` and `scikit-learn` to `requirements.txt` and install
2. Add `mlflow.db` and `mlruns/` to `.gitignore`
3. Create `features.py` and `train.py`, run `python train.py`
4. Start `mlflow ui --port 5050` and find your run
5. Change `max_depth` to 10 and run it again. Compare the MAE

Question: You commit a change to `train.py` without running it. How do you know which run used which version of the code?

Autologging

Writing `log_param` by hand for every parameter is tedious. Let MLflow do it:

```
mlflow.set_experiment("trip-duration")
mlflow.sklearn.autolog()

with mlflow.start_run():
    model = RandomForestRegressor(n_estimators=50, max_depth=10,
                                  random_state=42)
    model.fit(X_train, y_train)
    mean_absolute_error(y_test, model.predict(X_test))
```

On `fit()` MLflow logs all parameters, training metrics and the model. Metrics you compute afterwards with `sklearn.metrics` are logged too.

What autolog logs

18 parameters, not only the two we set:

```
n_estimators=50  max_depth=10  criterion=squared_error
min_samples_leaf=1  max_features=1.0  bootstrap=True  ...
```

Training metrics:

```
training_mean_absolute_error   3.26
training_r2_score              0.80
```

Plus the model, saved with `cloudpickle`.

Many runs: train_many.py

```
for max_depth in [2, 5, 10, 20]:
    with mlflow.start_run(run_name=f"rf-depth-{max_depth}"):
        params = {"n_estimators": 50, "max_depth": max_depth}
        mlflow.log_params(params)
        model = RandomForestRegressor(**params, random_state=42)
        model.fit(X_train, y_train)
        mae = mean_absolute_error(y_test, model.predict(X_test))
        mlflow.log_metric("mae", mae)
        mlflow.sklearn.log_model(
            model, name="model", input_example=X_test[:5],
            skops_trusted_types=["sklearn.tree._tree.Tree"],
        )
        print(f"max_depth={max_depth:2}  MAE: {mae:.2f} minutes")
```

Many runs

```
python train_many.py
```

```
max_depth= 2  MAE: 4.52 minutes
max_depth= 5  MAE: 3.81 minutes
max_depth=10  MAE: 3.44 minutes
max_depth=20  MAE: 3.32 minutes
```

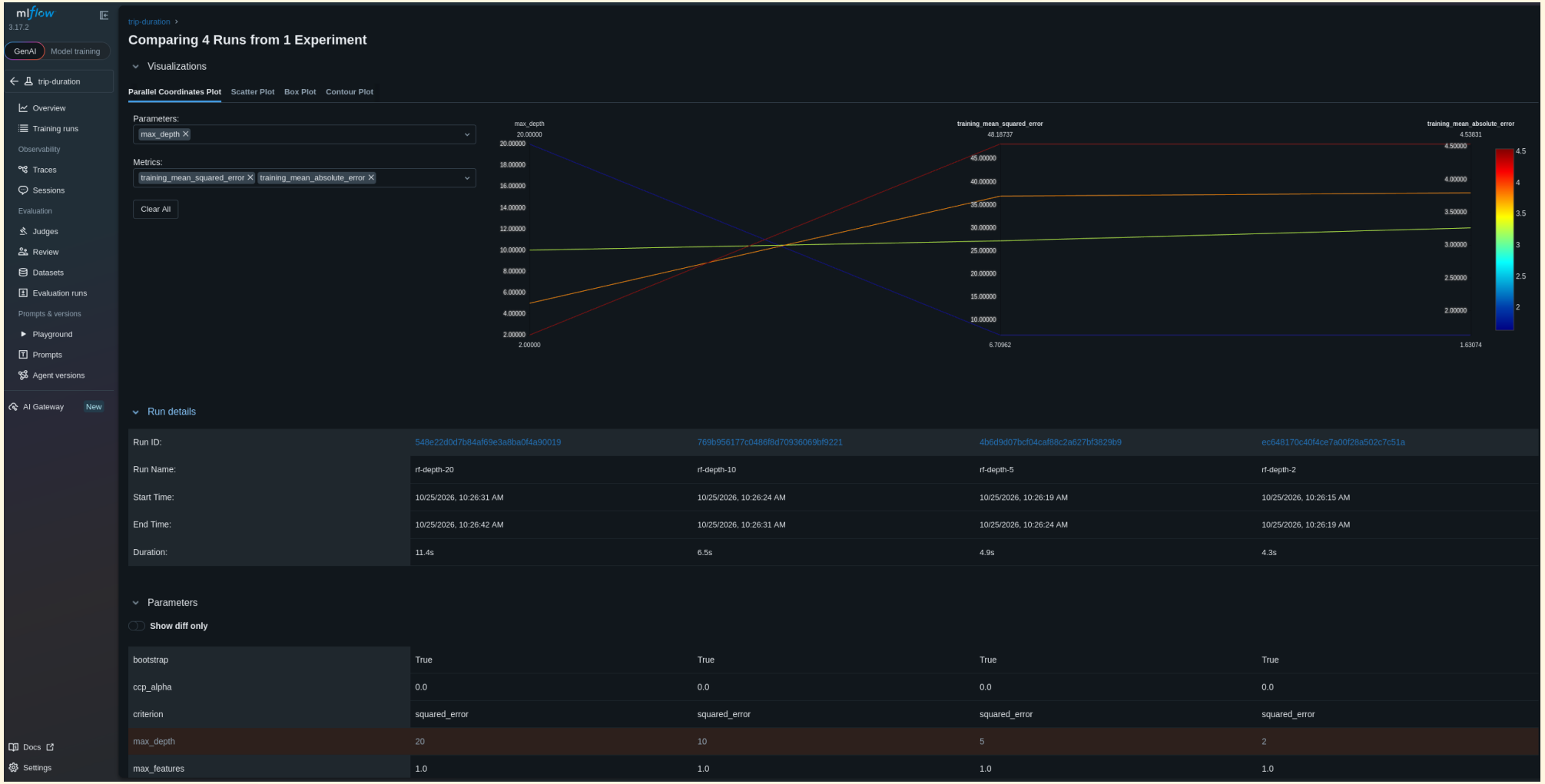
`run_name` gives each run a readable name in the UI instead of a random one.

Comparing runs in the UI

In the experiment view:

- **Table**: one row per run, add the `mae` column, sort by it
- Select runs → **Compare**: parameters side by side, metric charts
- **Chart view**: `mae` against `max_depth`

Comparing runs in the UI



Comparing runs from code

The UI is for people. Scripts can ask the same questions:

```
best = mlflow.search_runs(
    experiment_names=["trip-duration"],
    order_by=["metrics.mae ASC"],
    max_results=1,
).iloc[0]
print(best["tags.mlflow.runName"], best["metrics.mae"])
```

```
rf-depth-20 3.319029923858316
```

`search_runs` returns a pandas DataFrame: one row per run.

Is the best MAE the best model?

Run	MAE (min)	Model size
rf-depth-2	4.52	0.8 MB
rf-depth-5	3.81	1 MB
rf-depth-10	3.44	7 MB
rf-depth-20	**3.32**	**222 MB**

7 seconds better on average (0.12 min), 30 times bigger.

Exercise 2 - README - compare runs

1. Create `train_many.py` and run it
2. In the UI, sort the runs by `mae` and compare the four of them
3. Add `n_estimators` to the loop (`[10, 50]`): how many runs now?
4. Log the model size as a metric, and look at it in the chart view.
   Hint:
   `mlflow.log_metric("size_mb", len(pickle.dumps(model)) / 1e6)`
5. Find the best run with `mlflow.search_runs`

Part 3

A shared tracking server

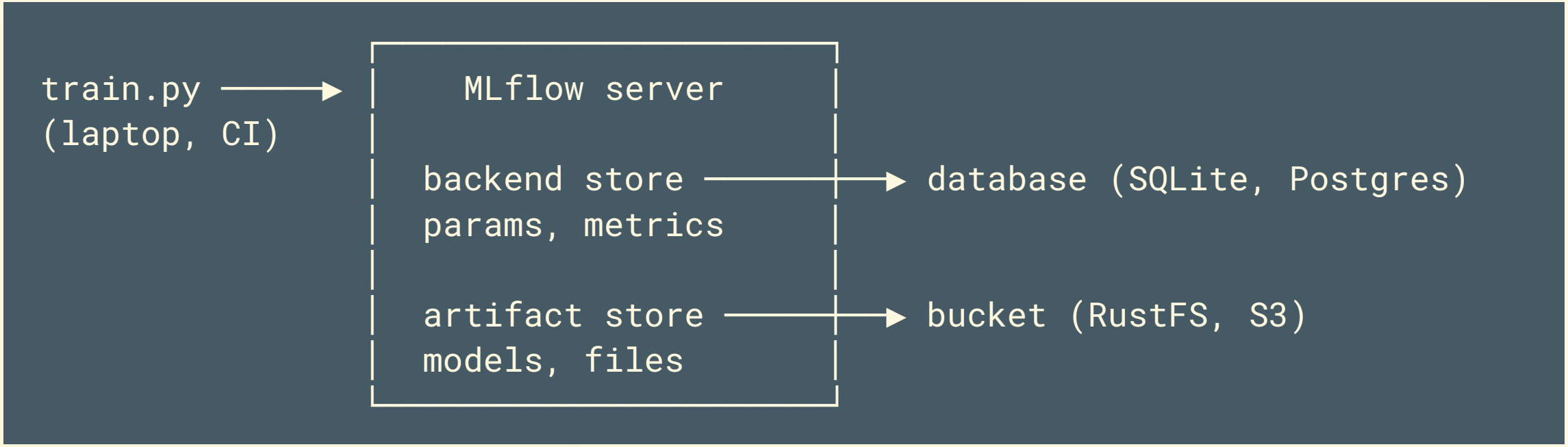
Why a server?

`mlflow.db` and `mlruns/` live on **your laptop**:

- Your teammate cannot see your runs
- Your CI job cannot log runs anywhere
- Your laptop dies, your experiments die with it

A **tracking server**: one place where the whole team, and CI, log runs and models.

Two stores



- **Backend store**: small, structured data. A database
- **Artifact store**: large files. Object storage, as in class 3

The server image

The official MLflow image does not include `boto3`, the library to talk to S3. Add it, as in class 2.
Create a Dockerfile under the server directory:

`server/Dockerfile`:

```
FROM ghcr.io/mlflow/mlflow:v3.17.2
RUN pip install --no-cache-dir boto3==1.40.70
```

compose.yaml

```yaml
services:
  storage:
    image: rustfs/rustfs:1.0.1
    ports: ["9000:9000", "9001:9001"]
    environment:
      RUSTFS_ACCESS_KEY: student
      RUSTFS_SECRET_KEY: student-secret
    volumes: [s3data:/data]

  mlflow:
    build: ./server
    ports: ["5050:5000"]
    environment:
      MLFLOW_S3_ENDPOINT_URL: http://storage:9000
      AWS_ACCESS_KEY_ID: student
      AWS_SECRET_ACCESS_KEY: student-secret
    command: >
      mlflow server --host 0.0.0.0 --port 5000
      --backend-store-uri sqlite:////mlflow/mlflow.db
      --artifacts-destination s3://mlflow
    volumes: [mlflowdb:/mlflow]
    depends_on: [storage]

volumes: {s3data: {}, mlflowdb: {}}
```

Start it

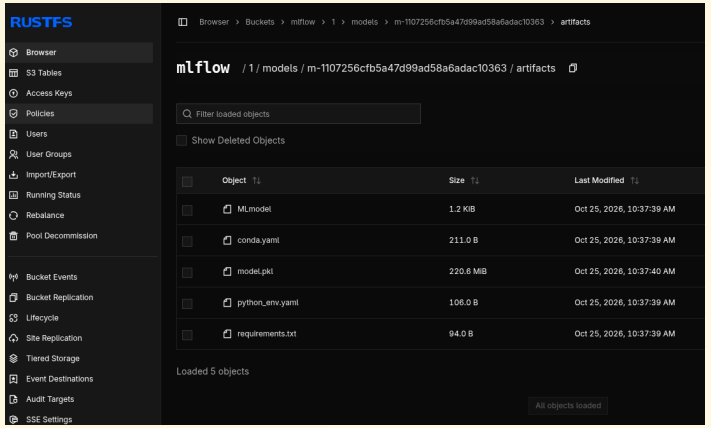
1. `docker compose up -d --build`
2. Open the RustFS console at http://localhost:9001 (*student / student-secret*) and manually create a bucket called `mlflow`
3. Open the MLflow UI at http://localhost:5050

Pointing your code at the server

No code changes. One environment variable:

```
export MLFLOW_TRACKING_URI=http://localhost:5050
python train_many.py
```

The runs show up on the server, the models in the `mlflow` bucket. Set it in **every terminal**: without it MLflow uses your local `mlflow.db` and does not find the server's runs and models.



Who has the credentials?

Your laptop only knows the server URL. **The server** talks to the bucket:

```
train.py ──HTTP──▶ MLflow server -- S3 + credentials ──▶ RustFS (or S3)
```

- Students, CI jobs and notebooks never see the bucket keys
- One place to rotate them
- Same rule as class 3: credentials in the environment, never in the image, never in Git

On a real team the server also needs a login. Ours has none.

Exercise 3 - README - tracking server

1. Create `server/Dockerfile` and `compose.yaml`
2. `docker compose up -d --build`, create the `mlflow` bucket
3. Set `MLFLOW_TRACKING_URI` and run `train_many.py`
4. Find the runs in the UI, and the model files in the RustFS console
5. `docker compose down` and `up` again. Are the runs still there? Why?

Part 4

Model Registry

From a run to a model

A logged model is identified by its run:

```
runs:/0ff1f7d3e8a64a8b9a2c.../model
```

Fine for you today. Not for the app that serves predictions:

- Which run is the good one? A random id says nothing
- A better model next week means changing the app's code

The **Model Registry** gives models a **name** and **versions**.

Registered models

```
Registered model: trip-duration
├── Version 1  ← run rf-depth-20      @champion
└── Version 2  ← run rf-depth-10
```

- **Name**: what the model does, chosen by you
- **Version**: a number, created each time you register a model
- **Alias**: a movable label pointing to one version

Each version remembers the run it came from: parameters, metrics, code.

Registering a model: register.py

```
import mlflow

best = mlflow.search_runs(
    experiment_names=["trip-duration"],
    order_by=["metrics.mae ASC"],
    max_results=1,
).iloc[0]

version = mlflow.register_model(f"runs:/{best.run_id}/model",
                                "trip-duration")
print("version", version.version)
```

```
Successfully registered model 'trip-duration'.
Created version '1' of model 'trip-duration'.
version 1
```

Or register while logging: `log_model(..., registered_model_name=...)`

Aliases

```
client = mlflow.MlflowClient()
client.set_registered_model_alias("trip-duration", "champion",
                                  version.version)
```

- `champion`: the model in production
- `challenger`: a candidate being evaluated

Promote a new model: move the alias. Roll back: move it back.

You may find **stages** (Staging, Production, Archived) in older tutorials: they are deprecated, use aliases.

Loading a model by name: predict.py

```
import mlflow
import pandas as pd

model = mlflow.pyfunc.load_model("models:/trip-duration@champion")

trips = pd.DataFrame({
    "trip_distance": [1.0, 10.0],
    "pickup_hour": [8.0, 3.0],
    "pickup_weekday": [0.0, 6.0],
    "PULocationID": [161.0, 132.0],
    "DOLocationID": [237.0, 230.0],
})
print(model.predict(trips))
```

```
[ 8.8166715  22.70563524]
```

pyfunc

`mlflow.pyfunc.load_model` gives you the same interface for any model:

```
model.predict(dataframe)
```

scikit-learn, XGBoost, PyTorch: the code that loads and calls the model does not change.

The app does not know which run, which parameters or which library. It asks for `trip-duration@champion`.

Serving a model

A REST API in one command, no Flask code:

```
mlflow models serve -m "models:/trip-duration@champion" \
  -p 5002 --env-manager local
```

```
INFO:     Uvicorn running on http://127.0.0.1:5002
```

- `--env-manager local`: use the libraries of your virtual environment. Without it MLflow builds a new one from the model's `requirements.txt`
- Two endpoints: `GET /ping` and `POST /invocations`

Calling the model

```
curl -X POST http://localhost:5002/invocations \
  -H "Content-Type: application/json" \
  -d '{"dataframe_records": [
        {"trip_distance": 1.0, "pickup_hour": 8, "pickup_weekday": 0,
         "PULocationID": 161, "DOLocationID": 237},
        {"trip_distance": 10.0, "pickup_hour": 3, "pickup_weekday": 6,
         "PULocationID": 132, "DOLocationID": 230}]}'
```

```
{"predictions": [8.816671500721501, 22.705635238095233]}
```

A short trip in the morning: 9 minutes. 10 miles at 3am: 23 minutes.

A new champion

1. Register the `rf-depth-10` run as version 2
2. Move `champion` to version 2
3. Restart `mlflow models serve`

Same command, same URL, new model. The app calling `/invocations` does
not change.

Question: The new model is worse in production. How do you roll back, and how long does it take?

Exercise 4 - README - register and serve

1. Create `register.py`: register the best run, set the `champion` alias
2. Find the model in the UI, under **Models**
3. Create `predict.py` and load `trip-duration@champion`
4. Serve it with `mlflow models serve` and call it with `curl`
5. Register `rf-depth-10`, move `champion` to it, restart and call again

Part 5

Data versioning

What about the data?

MLflow knows the code, parameters and model. The data?

- Git is for code: a 50 MB Parquet file, ten versions, 500 MB repository
- The bucket keeps the **latest** file. Overwrite it and the old one is gone
- "Trained on `trips.parquet`" says nothing if the file changed since

We need versions of the data, linked to the Git commit.

DVC = Data Version Control



Git stores a small `.dvc` file with a hash. The data goes to a remote: a bucket.

DVC with a bucket as remote

```
pip install "dvc[s3]"
dvc init
dvc add data/trips.parquet
git add data/trips.parquet.dvc data/.gitignore
git commit -m "Track trips data with DVC"

dvc remote add -d storage s3://dvc
dvc remote modify storage endpointurl http://localhost:9000
dvc remote modify --local storage access_key_id student
dvc remote modify --local storage secret_access_key student-secret
dvc push
```

A teammate clones the repo and runs `dvc pull`: same data, same hash.

The .dvc file

What Git sees:

```
outs:
- md5: 3fbbab2b8949308a27b6285f36aae825
  size: 60840464
  hash: md5
  path: trips.parquet
```

- New data: `dvc add` again, commit the new `.dvc` file
- Go back: `git checkout <commit>` then `dvc checkout`

Code version and data version in the same commit.

Exercise 5 - README - DVC (if time allows)

If we run out of time, do it at home:

1. Install `dvc[s3]` and run `dvc init` in your repository
2. Create a `dvc` bucket in the RustFS console
3. `dvc add data/trips.parquet` and commit the `.dvc` file. `data/` cannot stay in `.gitignore`: DVC refuses ignored folders
4. Add the RustFS remote and `dvc push`. Find the data in the bucket
5. Delete `data/trips.parquet` and get it back with `dvc pull`

Hugging Face Hub

The GitHub of models and datasets:

- Thousands of public **datasets** and pretrained **models**
- Versioned with Git, large files stored outside
- `pip install datasets` and load one in a line

Good for sharing **public** data and models. For your team's private data: a bucket and DVC.

Wrap-up

What	Where it lives	Class
Code	Git, GitHub	1
Environment	Docker	2
Data	Bucket, Parquet, DVC	3, 5
Checks	GitHub Actions	4
Experiments	MLflow tracking	5
Models	MLflow Model Registry	5

Your project in the sprints uses all of them.

If you want to learn more...

Extra material

The MLmodel file

Every logged model has one. It says how to load it:

```
flavors:
  python_function:
    loader_module: mlflow.sklearn
    python_version: 3.14.7
  sklearn:
    serialization_format: skops
    sklearn_version: 1.9.1
signature:
  inputs: '[{"type": "double", "name": "trip_distance"}, ...]'
  outputs: '[{"type": "tensor", ...}]'
```

Flavors: the same model loaded as scikit-learn or as generic pyfunc.

Logging from CI

Train in a GitHub Actions job (class 4) and log to the team server:

```
      - run: python train_many.py
        env:
          MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_TRACKING_URI }}
```

Every merge to `main` produces a run, linked to its commit.

The server must be reachable from GitHub: a laptop server is not.

DVC pipelines

Describe the steps in `dvc.yaml`:

```
stages:
  train:
    cmd: python train.py
    deps:
      - data/trips.parquet
      - features.py
      - train.py
    outs:
      - model.skops
```

`dvc repro` reruns only the stages whose inputs changed.